Search Keyword Revenue Pipeline

Technical Documentation

Language	Python 3.10 / PySpark 3.3
Runtime	AWS Glue 4.0
Storage	Amazon S3 (analytics-hitdata-s3)
Source Files	glue_job.py + metadata_profiler.py + metadata_config.json
Version	2.0
Last Updated	2026-03-13

1. Purpose

This document provides a complete technical reference for the Search Keyword Revenue Pipeline. It covers the business problem, design of each component, all configuration options, the data model, the attribution model, known limitations, and guidance for extending the system.

2. Business Problem

The client (esshopzilla.com) uses Adobe Analytics on their e-commerce site. Hit-level data is exported as tab-delimited files to S3. The client wants to know:

- How much revenue is being driven by external search engines such as Google, Yahoo, and Bing?
- Which specific search keywords are delivering the highest revenue?

The pipeline answers this by attributing each purchase to the most recent external search engine referral from the same visitor IP (last-touch attribution) and aggregating total revenue by search engine domain and keyword.

3. Source Data Format

Source files are tab-delimited text files with a header row. Each row represents a single hit from a visitor. Fields containing commas or newlines are quoted with double-quotes.

Column	Type	Description
hit_time_gmt	int	Unix timestamp of the hit.
date_time	datetime	Human-readable timestamp in the report suite timezone. Used for last-touch ordering.
user_agent	text	HTTP User-Agent string. May contain commas — fields are double-quoted in source.
ip	string	Visitor IP address. Used as the session proxy for last-touch attribution.
geo_city	string	City derived from IP geolocation.
geo_country	string	Country abbreviation.
geo_region	string	State or region derived from IP.
pagename	string	Page name dimension. Falls back to page_url if empty.
page_url	string	URL of the hit.
product_list	text	Comma-delimited products. Each: category;name;qty;revenue;events;evar. Revenue = 4th semicolor
referrer	string	Referring URL. Used to detect search engine and extract keyword.

Column	Type	Description
event_list	text	Comma-separated event codes. 1=Purchase, 2=Product View, 12=Cart Add. Revenue only counted v

4. Component Reference

4.1 glue_job.py

The main pipeline entry point. Contains three primary components: module-level constants/helpers, MetadataProfiler class, and SearchKeywordRevenueProcessor class.

Module-Level Constants

Constant	Purpose
BUCKET	S3 bucket name. Change here to deploy to a different bucket.
PROCESSED_PREFIX	S3 prefix for analysis output files.
METADATA_PREFIX	S3 prefix for schema profiling JSON reports.
LOGS_PREFIX	S3 prefix for run summaries and pipeline logs.
CHECKSUMS_PREFIX	S3 prefix for the idempotency checksum registry.
CONFIG_FILE	Local filename for metadata config (copied via --extra-files at job start).
SEARCH_ENGINE_PATTERNS	List of domain substrings identifying search engines. Add entries to extend coverage without touching bu
KEYWORD_PARAMS	Maps engine family to query-string param: Yahoo uses p=, all others use q=.

MetadataProfiler Class

Validates and profiles the source DataFrame against the expected schema in metadata_config.json.

Method	Description
__init__(expected_columns)	Parses expected column definitions. Accepts plain string lists and dict lists with name/type keys.
_detect_mismatches(columns)	Compares actual vs expected columns. Returns list of mismatch dicts: missing_columns, extra_colun
profile(df, source_key)	Runs two distributed aggregation passes. Pass 1: min/max/null counts. Pass 2: countDistinct. Both c

SearchKeywordRevenueProcessor Class

Implements the business logic for search keyword revenue attribution.

Method	Description
_build_search_engine_column(df)	Chains F.when() expressions against SEARCH_ENGINE_PATTERNS on the lowercased referrer do
_build_keyword_column(df)	Extracts keyword via regex: Yahoo matches p=[^&]+, all others match q=[^&]+. URL-encoded + repla

Method	Description
_build_revenue_column(df)	Uses Spark aggregate() higher-order function. Sums 4th semicolon field per product in product_list. tr
transform(df)	Full pipeline: enrich df, split into search_touchpoints + purchases, left-join on IP (touchpoint before pu

4.2 metadata_profiler.py

A standalone version of MetadataProfiler that reads directly from a local file using Python's csv.DictReader. Intended for local development and testing. The Glue job uses the Spark-native version in glue_job.py.

Feature	Detail
Memory model	Row-by-row streaming via csv.DictReader. Memory usage is O(unique values per column), not O(total rows).
Null handling	NULL_SENTINELS class constant: empty string, 'null', 'none', 'na', 'n/a' (case-insensitive).
API	profile(source_bucket, source_key) returns the same report dict structure as the Spark version.

4.3 metadata_config.json

JSON file defining the expected schema for source hit-level files. Passed to Glue via --extra-files.

```
{
"file_type": "delimited_text",
"delimiter": "\t",
"expected_columns": [
{"name": "hit_time_gmt", "type": "int"},
{"name": "date_time", "type": "datetime"},
...
]
}
```

NOTE

Type information is recorded in the metadata report but does not affect parsing — all values are read as strings regardless of declared type.

5. Revenue Attribution Model

The pipeline uses a last-touch attribution model scoped to visitor IP address:

1. Rows where the referrer is a recognised external search engine are identified as search touchpoints. Domain and keyword are extracted from the referrer URL.
2. Rows where event_list contains purchase event code '1' and product_list revenue > 0 are purchase events.
3. Each purchase is joined to all search touchpoints from the same IP address that occurred at or before the purchase timestamp.
4. A Window function (row_number, partitioned by IP + purchase time + revenue, ordered by touchpoint time desc) selects the most recent touchpoint as the attributed referrer.
5. Purchases with no preceding search touchpoint are excluded from the output.
6. Revenue is aggregated by (search_engine_domain, search_keyword) and rounded to 2 decimal places.

NOTE

IP address is used as a session proxy — multiple visitors behind the same NAT may be conflated.

Last-touch gives 100% revenue credit to the most recent search click before a purchase.

Each purchase in a session is attributed independently to its own most recent search touchpoint.

6. Output File Format

UTF-8 tab-delimited file with header row, sorted descending by Revenue:

Column	Type	Description
Search Engine Domain	string	Lowercased domain from referrer. E.g. google.com, bing.com, search.yahoo.com.
Search Keyword	string	Lowercased keyword from q= or p= query param. URL-encoded + replaced with spaces.
Revenue	decimal	Total revenue in dollars, summed across all attributed purchases. Rounded to 2 decimal places.

Example output:

```
Search Engine Domain Search Keyword Revenue
www.google.com ipod 540.00
www.bing.com zune 250.00
```

NOTE

Revenue only includes rows where event code '1' (Purchase) is present in event_list.

7. Idempotency Design

A SHA-256 content checksum registry in S3 prevents reprocessing the same source file multiple times. The checksum is computed by streaming the file in 8 MB chunks via boto3 — constant memory regardless of file size.

The --force true argument bypasses this check. The registry entry is then overwritten by the new run.

8. Error Handling

The top-level try/except wraps the entire main() call. On any unhandled exception:

1. The error message is captured and logged to PIPELINE_LOG_LINES.
2. A FAILED run summary JSON is written to logs/YYYY/MM/DD/run-summary-.json.
3. The accumulated pipeline log is written to logs/YYYY/MM/DD/pipeline-.log.
4. The original exception is re-raised so AWS Glue marks the run as FAILED.

S3 keys are constructed once before main() is called, ensuring the FAILED summary and log always land at the same timestamped paths the successful run would have used.

9. Configuration Reference

Constant / Argument	Location	How to Change
BUCKET	glue_job.py line 28	Edit constant in glue_job.py and re-upload the script to S3.
SEARCH_ENGINE_PATTERNS	glue_job.py line 37	Add or remove domain substrings. No other code changes required.
Expected schema	metadata_config.json	Edit file and re-upload to S3. Glue reads it via --extra-files on next run.
--force	Job argument	Pass at job invocation. Not persisted.
spark.sql.shuffle.partitions	glue_job.py line 98	Increase for large files (default 200). Rule of thumb: 2–4x the number of vCPU
coalesce(1)	glue_job.py Step 4	Remove for 10 GB+ files. Write partitioned output and use S3 Batch Ops to re

10. Local Testing

Run the standalone profiler against any tab-delimited file without an AWS connection:

```
from metadata_profiler import MetadataProfiler
import json
profiler = MetadataProfiler(
input_file='sample_data.sql',
delimiter='\t',
expected_columns=[
{"name": "hit_time_gmt", "type": "int"},
{"name": "date_time", "type": "datetime"}
]
)
report = profiler.profile('local', 'sample_data.sql')
print(json.dumps(report, indent=2))
```

To test the full Spark business logic locally:

```
export PYSPARK_PYTHON=python3
pyspark --master local[*]
```

11. Known Limitations

Limitation	Detail / Mitigation
coalesce(1) bottleneck	Single-executor write is safe up to ~2 GB. For 10 GB+: remove coalesce(1) and implement S3 Batch Op
IP as session proxy	Multiple visitors behind the same NAT/proxy IP may have sessions conflated. A proper visitor ID column

Limitation	Detail / Mitigation
Last-touch only	100% credit to most recent search click. First-touch, linear, or data-driven models require additional attrib
Partial URL-decoding	Only + is replaced with space. Full percent-decoding (%20, %2B, etc.) not applied. A UDF or regex could
Hardcoded search engines	SEARCH_ENGINE_PATTERNS constant controls coverage. Adding new engines requires a code chang
Output overwrites same day	Re-running with --force on the same calendar day overwrites the processed .tab file. Logs and metadata